



JavaScript Loops & Iteration

Loops and iteration are fundamental for processing data in JavaScript. This cheat sheet covers the most important loop types and iteration patterns for arrays, objects, and more.

💡 Tip

Complete Dev Roadmap with SaaS Boilerplate at thedevspace.io/

for loop

```
1 // logs numbers 0 through 4
2 for (let i = 0; i < 5; i++) {
3   console.log(i);
4 }
```

- Use `for` when you know the iteration count
- Use `let` for loop counters to keep scope local
- Prefer small loop bodies for readability and performance
- For heavy work, break tasks into chunks to avoid blocking

while loop

```
1 // while loop example
2 let j = 0;
3 while (j < 5) {
4   console.log(j);
5   j++;
6 }
```

- Use `while` when the number of iterations depends on a runtime condition
- Ensure loop conditions will eventually become false to avoid infinite loops
- Keep side effects clear and local to the loop body

do...while loop

```
1 // do...while runs the body at least once
2 let k = 0;
3 do {
4   console.log(k);
5   k++;
6 } while (k < 5);
```

- Use `do...while` when you must run the body at least once before checking the condition
- Prefer explicit checks and simple bodies for clarity

for...of

```
1  const arr = [10, 20, 30];
2  // iterate values from an iterable
3  for (const value of arr) {
4    console.log(value);
5  }
```

- Use `for...of` for arrays, strings, maps, sets and other iterables
- Works with generators and custom iterators
- Avoid mutation of the iterated collection while iterating

for...in

```
1 const obj = { a: 1, b: 2 };
2 // iterate keys on an object (includes inherited
  keys)
3 for (const key in obj) {
4   if (Object.prototype.hasOwnProperty.call(obj,
    key)) {
5     console.log(key, obj[key]);
6   }
7 }
```

- Use `for...in` for enumerating object keys, but filter inherited properties
- Prefer `Object.keys` / `Object.entries` for predictable key ordering

Array Iteration Methods

```
1 // functional iteration examples
2 arr.forEach((x) => console.log(x));
3 const doubled = arr.map((x) => x * 2);
4 const filtered = arr.filter((x) => x > 15);
5 const sum = arr.reduce((a, b) => a + b, 0);
```

- `map` returns a new transformed array
- `forEach` is for side effects and returns undefined
- `filter` creates a new array with matching items
- `reduce` aggregates values into a single result
- Prefer pure functions in callbacks to keep code predictable

Breaking and Continuing

```
1  for (let i = 0; i < 10; i++) {  
2    // exit loop when i is 5  
3    if (i === 5) break;  
4  
5    // skip even numbers  
6    if (i % 2 === 0) continue;  
7  
8    console.log(i);  
9  }
```

- **break** exits the nearest loop immediately
- **continue** skips the rest of the current iteration
- Consider early returns from functions instead of nested loops

Iterators and Generators

```
1 function* gen() {  
2   yield 1;  
3   yield 2;  
4 }  
5 const it = gen();  
6 console.log(it.next().value);  
7 console.log(it.next().value);
```

- Generators pause with `yield` and resume on next invocation
- Use generators to model lazy sequences and custom iteration
- Consume generators with `for...of` or by calling `next()` explicitly
- Generators can simplify async stream processing when combined with async iterators